Introducción
Attention is all you need
Transformers / BERT / LLMs
Large Language Models
Conceptos Importantes
LLMs como modelos de Machine Learning
Agentes
Agentes

Procesamiento de Lenguaje Natural

De BERT a LLMs: Explorando los Límites del Procesamiento de Lenguaje Natural

Mauricio Toledo-Acosta
mauricio.toledo@unison.mx

Departamento de Matemáticas
Universidad de Sonora

Section 1

Introducción

Desventajas de los modelos anteriores

- ¿Cómo toman en cuenta el contexto de una palabra modelos como `Word2Vec`, `FastText`, `GloVe`?
- ¿Cómo son los embeddings de *banco*? ¿cómo podemos considerar las diferentes acepciones?
- ¿Cómo contribuyen los embeddings de palabras para crear embeddings de documentos? ¿se considera el orden? ¿las relaciones entre palabras?

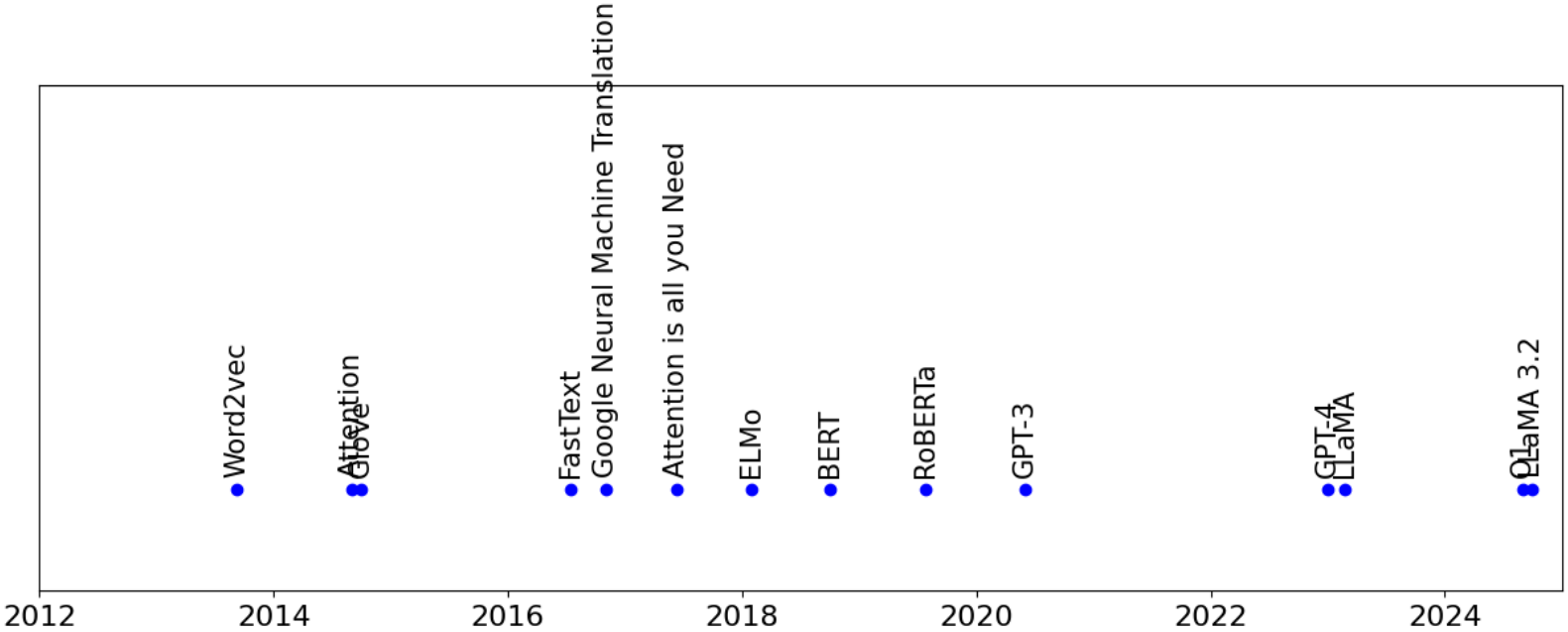
Machine Translation

One naturally wonders if the problem of translation could conceivably be treated as a problem in cryptography. When I look at an article in Russian, I say: 'This is really written in English, but it has been coded in some strange symbols. I will now proceed to decode.

Warren Weaver, 1947

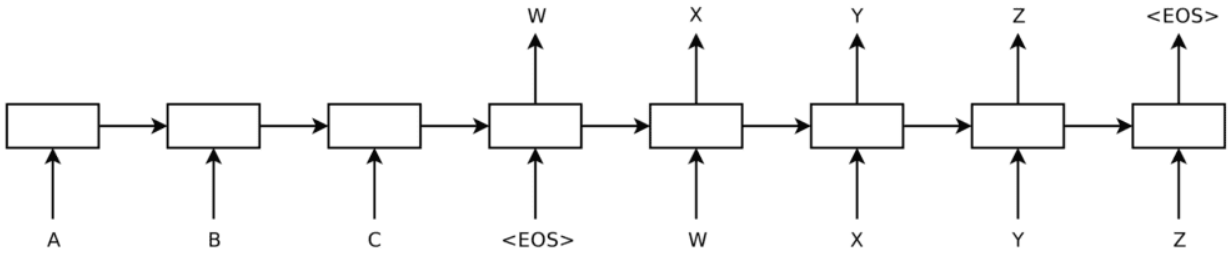
- Rule-based translation (1970-1990).
- Statistical machine translation (1990-2010).
- Deep learning-based translation (2003-).

Timeline

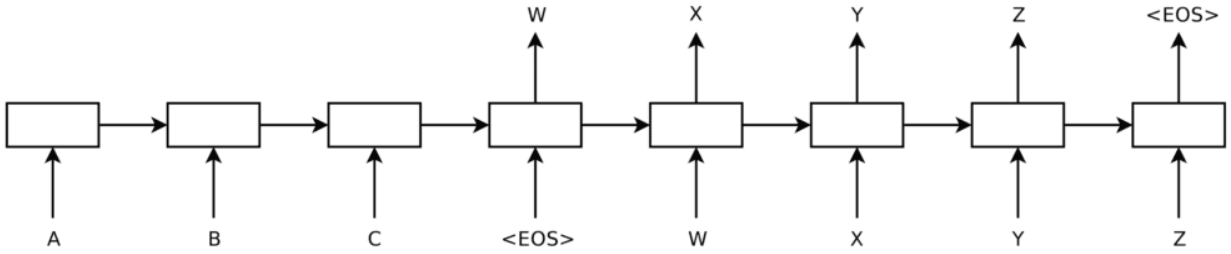

Word2vec
Attention
Glove
FastText
Google Neural Machine Translation
Attention is all you Need
ELMo
BERT
RoBERTa
GPT-3
GPT-4
LLaMA
LLaMA 3.2
2012
2014
2016
2018
2020
2022
2024

Seq2Seq

- Seq2Seq es una familia de enfoques de ML en el NLP para: traducción de idiomas, generación de subtítulos para imágenes, modelos conversacionales y resumen de textos.
- Convierten una secuencia de entrada en otra secuencia de salida.

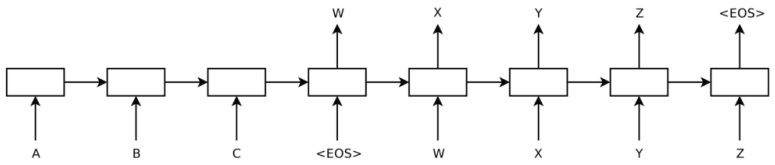


Seq2Seq



- El modelado y la generación de secuencias se realizaron mediante arquitecturas RNN. Esto llevo al problema del desvanecimiento de gradiente.
- La arquitectura LSTM se convirtió en la estrategia estándar para el modelado de secuencias largas hasta la aparición en 2017 de los Transformers.
- Las RNN operan un token a la vez, del primero al último; no pueden operar en paralelo con todos los tokens de una secuencia.

Seq2Seq



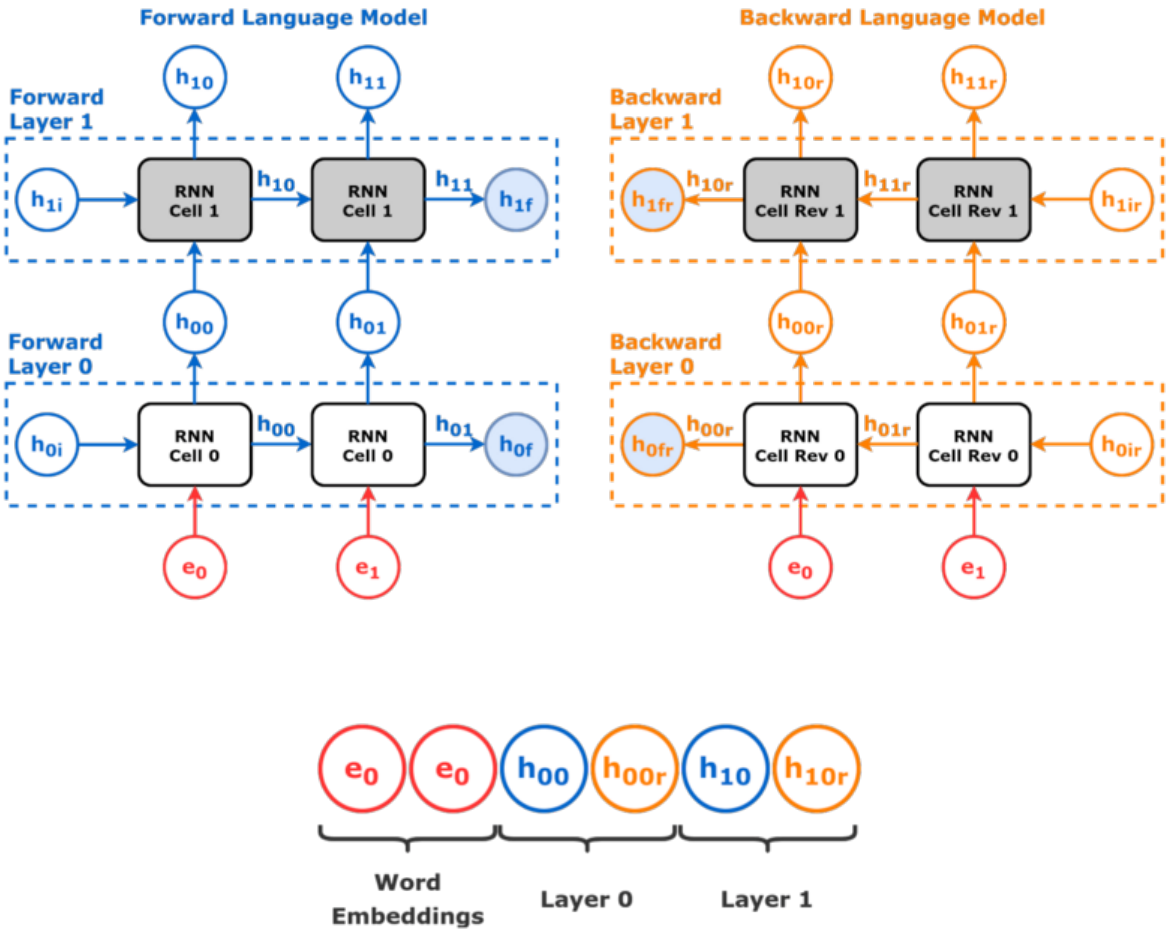
- Los primeros modelos seq2seq carecían de mecanismo de atención.
- El vector de estado sólo es accesible después de procesar la última palabra del texto de origen.
- Este vector conserva la información sobre toda la frase original, en la práctica la información se conserva mal, ya que la entrada es procesada secuencialmente y si la entrada es larga, el vector de salida no podría contener toda la información relevante.

ELMo

- ELMo (embeddings from language model) es un modelo de embeddings de secuencias de palabras.
- La arquitectura de ELMo logra una comprensión contextual de los tokens.
- ELMo es un LSTM bidireccional multicapa sobre una capa de embeddings de tokens. La salida de todas las LSTM concatenadas consiste en la incrustación de tokens.

Artículo

ELMo


Forward Language Model
Backward Language Model
Forward Layer 1
Forward Layer 0
Backward Layer 1
Backward Layer 0
RNN Cell 1
RNN Cell 0
RNN Cell Rev 1
RNN Cell Rev 0
Word Embeddings
Layer 0
Layer 1

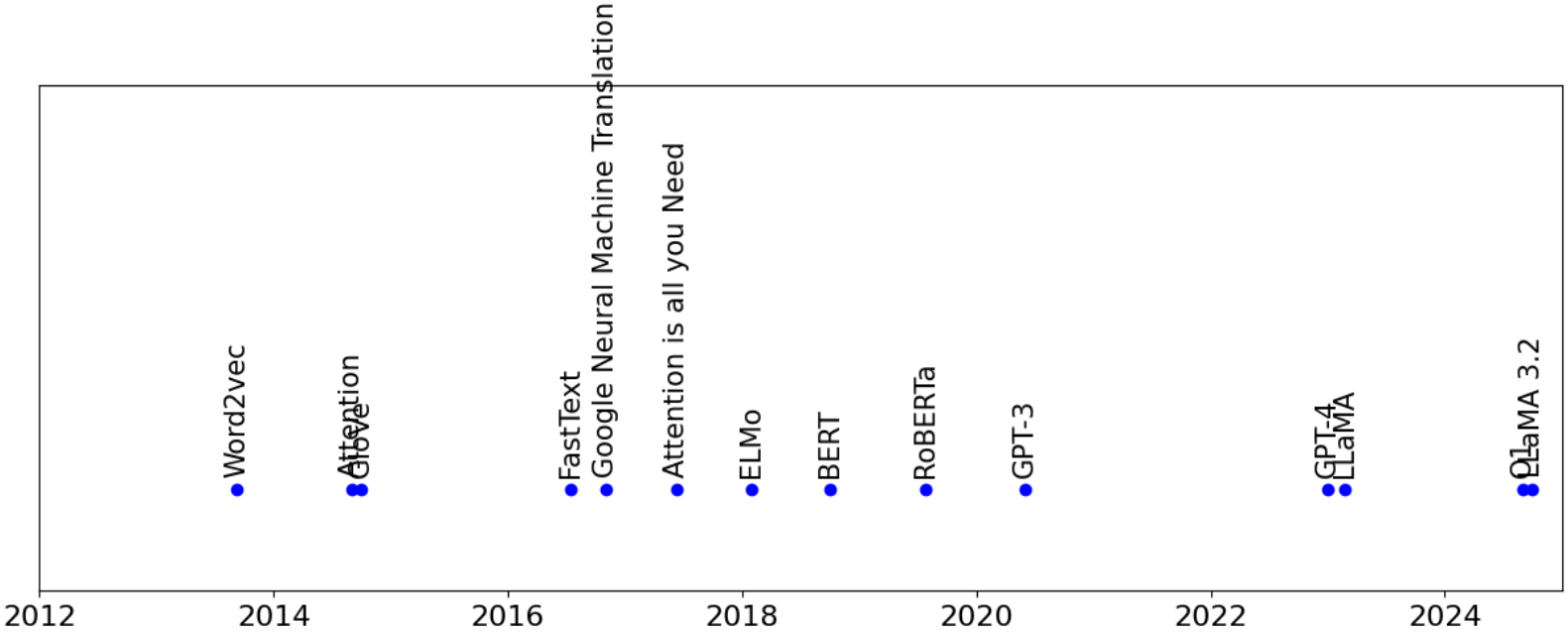
Mecanismos de Atención, 2014

- El **mecanismo de atención** es una mejora introducida en 2014 para abordar las limitaciones de la arquitectura básica Seq2Seq.
- Permite al modelo centrarse en diferentes partes de la secuencia de entrada durante el proceso de decodificación. Es un mecanismo que permite que los tokens *hablen* entre sí.
- En 2016, Google Translate se actualizó con Google Neural Machine Translation, que reemplazó el modelo basado en *statistical machine translation*. GNMT fue un Seq2Seq donde el codificador y decodificador contenian 8 capas de LSTMs bidireccionales.

Mecanismo de Atención

- Hay dos tipos de atención: Self Attention & Cross Atention.
- Uszkoreit: La atención sin recurrencia es suficiente para la traducción lingüística.

Timeline


Word2vec
Attention
Glove
FastText
Google Neural Machine Translation
Attention is all you Need
ELMo
BERT
RoBERTa
GPT-3
GPT-4
LLaMA
LLaMA 3.2
2012
2014
2016
2018
2020
2022
2024

Section 2

Attention is all you need

Attention is all you need, 2017

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

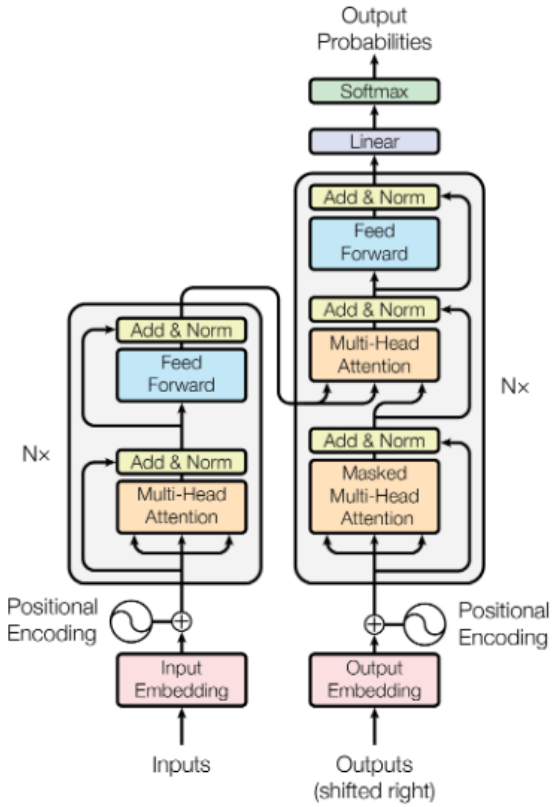
Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Artículo

El transformador

- Nueva arquitectura llamada **transformador**, basada en un mecanismo de atención multi-head (atención en paralelo).
- Los transformadores tienen la ventaja de no tener unidades recurrentes, por lo que requieren menos tiempo de entrenamiento que las arquitecturas RNN.
- Fué desarrollado para la traducción, encontró aplicaciones en los LLM, visión computacional, audio, etc.



Mecanismo de atención

1. Atención de una cabeza

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right) V_i$$

2. Multi-cabeza: concatenación y proyección

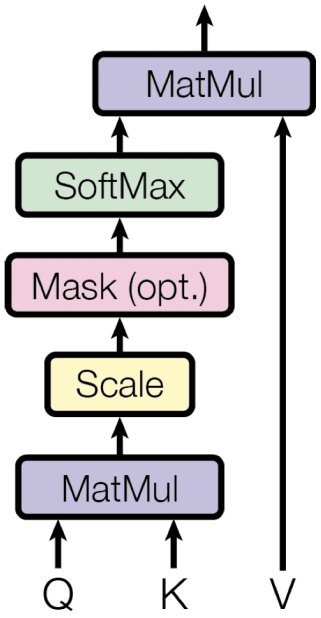
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)$$

3. Conexión y normalización

$$\text{out} = \text{LayerNorm}(x + \text{MultiHead}(x))$$

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



BERT, 2018

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

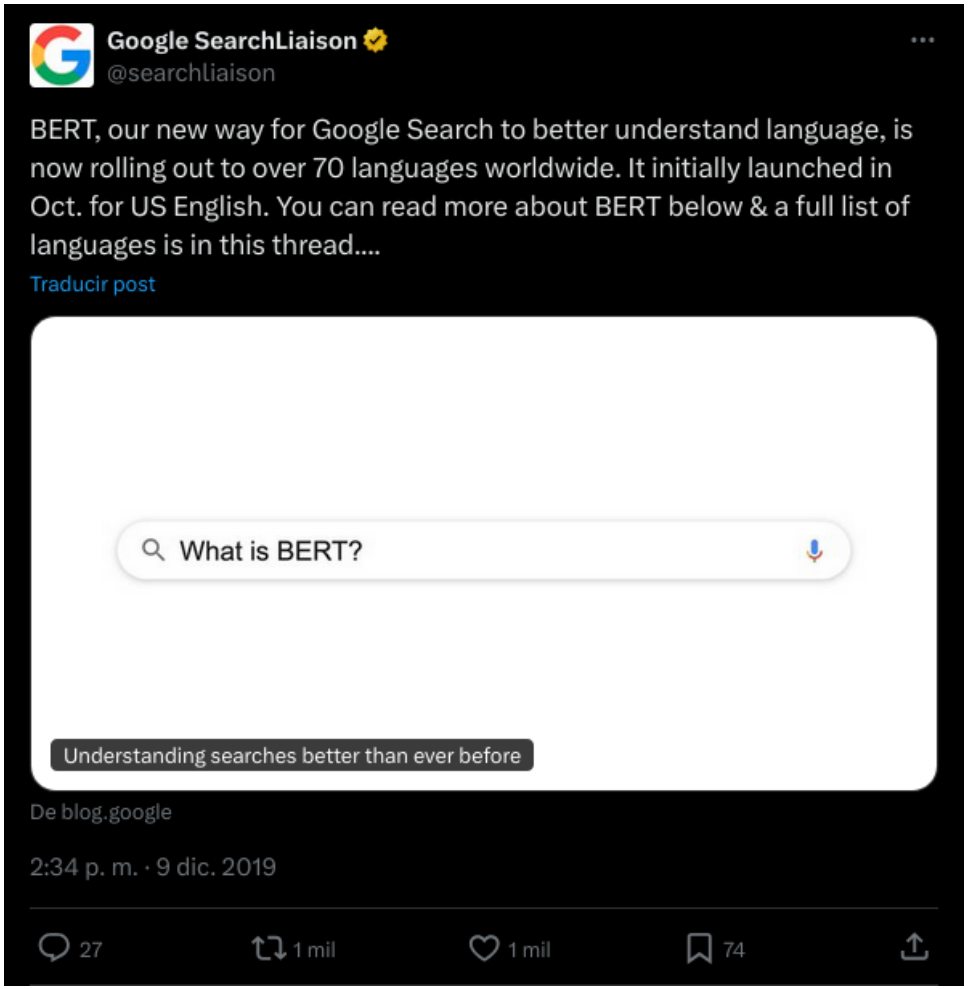
Jacob Devlin **Ming-Wei Chang** **Kenton Lee** **Kristina Toutanova**
Google AI Language
{jacobdevlin,mingweichang,kentonl,kristout}@google.com

Hubo dos implementaciones:

- $\text{BERT}_{\text{BASE}}$ (110 million parameters)
- $\text{BERT}_{\text{LARGE}}$ (340 million parameters)

Artículo, repositorio

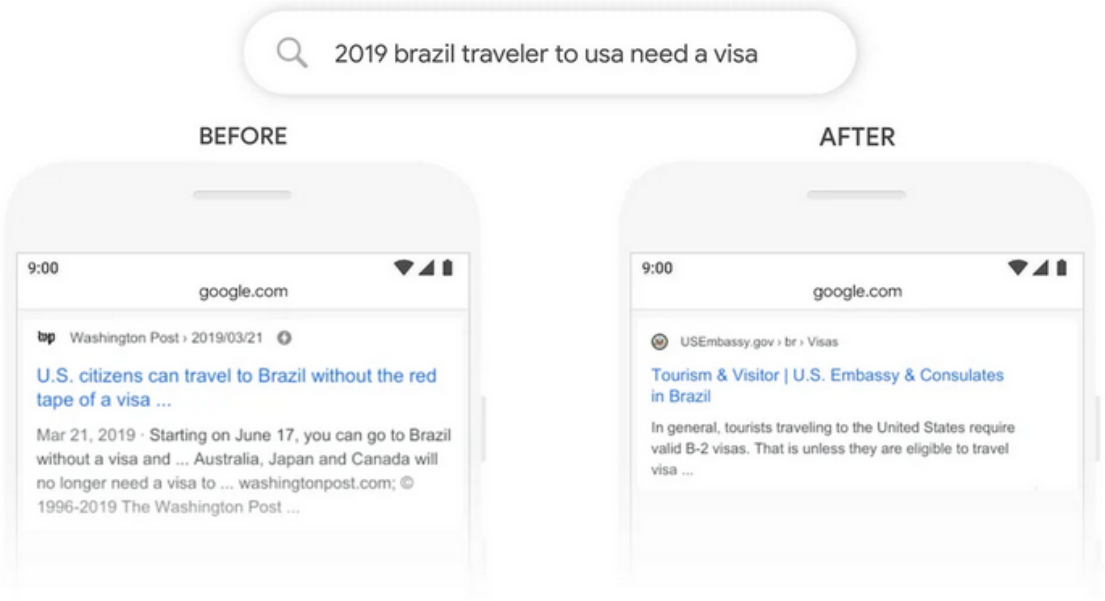
BERT: A blast from the past



El anuncio, noticias 1, noticias 2

Ventajas de BERT

Here's a search for "2019 brazil traveler to usa need a visa." The word "to" and its relationship to the other words in the query are particularly important to understanding the meaning. It's about a Brazilian traveling to the U.S., and not the other way around. Previously, our algorithms wouldn't understand the importance of this connection, and we returned results about U.S. citizens traveling to Brazil. With BERT, Search is able to grasp this nuance and know that the very common word "to" actually matters a lot here, and we can provide a much more relevant result for this query.



BERT en las busquedas

Variantes de BERT

- RoBERTa (2019): A Robustly Optimized BERT Pretraining Approach.
- DistilBERT (2019): A distilled version of BERT: smaller, faster, cheaper and lighter. huggingface
- CamemBERT (2020): Una variante de RoBERTa entrenada en un corpus francés.
- BETO (2019): Una variante de BERT en español.

A partir de 2018, comenzó la serie de modelos GPT de Transformers *decoder only* de OpenAI.

BERT: Arquitectura

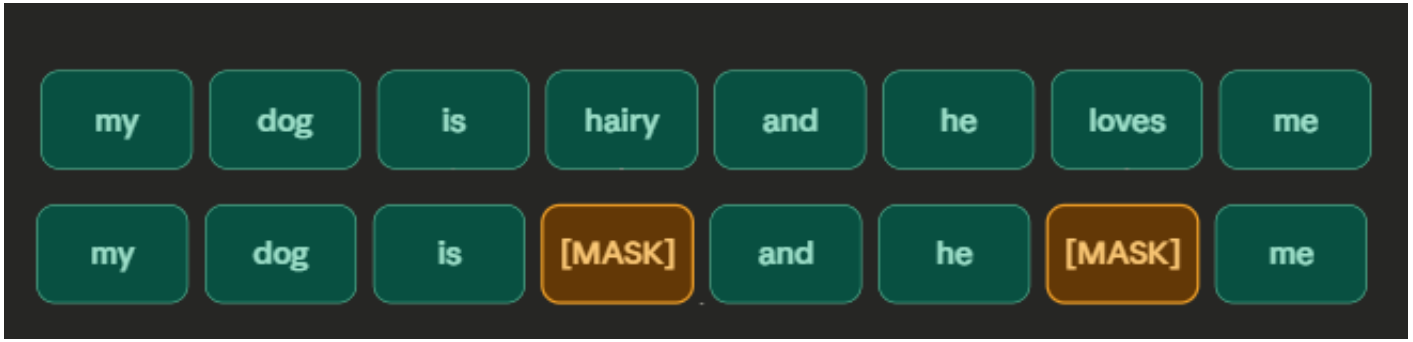
BERT es una arquitectura de transformador *encoder only*. BERT consta de 4 partes:

- **Tokenizador**: Este módulo convierte un texto en una secuencia de índices (tokens).
- **Embeddings**: Este módulo convierte la secuencia de tokens en una matriz de embeddings que representan los tokens.
- **Encoder**: Un stack de bloques de Transformers con autoatención, sin masking.
- **Task head**: Este módulo convierte de los embeddings finales en tokens mediante una una distribución de probabilidad. Esta cabeza se puede cambiar.

Tareas de entrenamiento

BERT se pre-entrena simultáneamente en dos tareas:

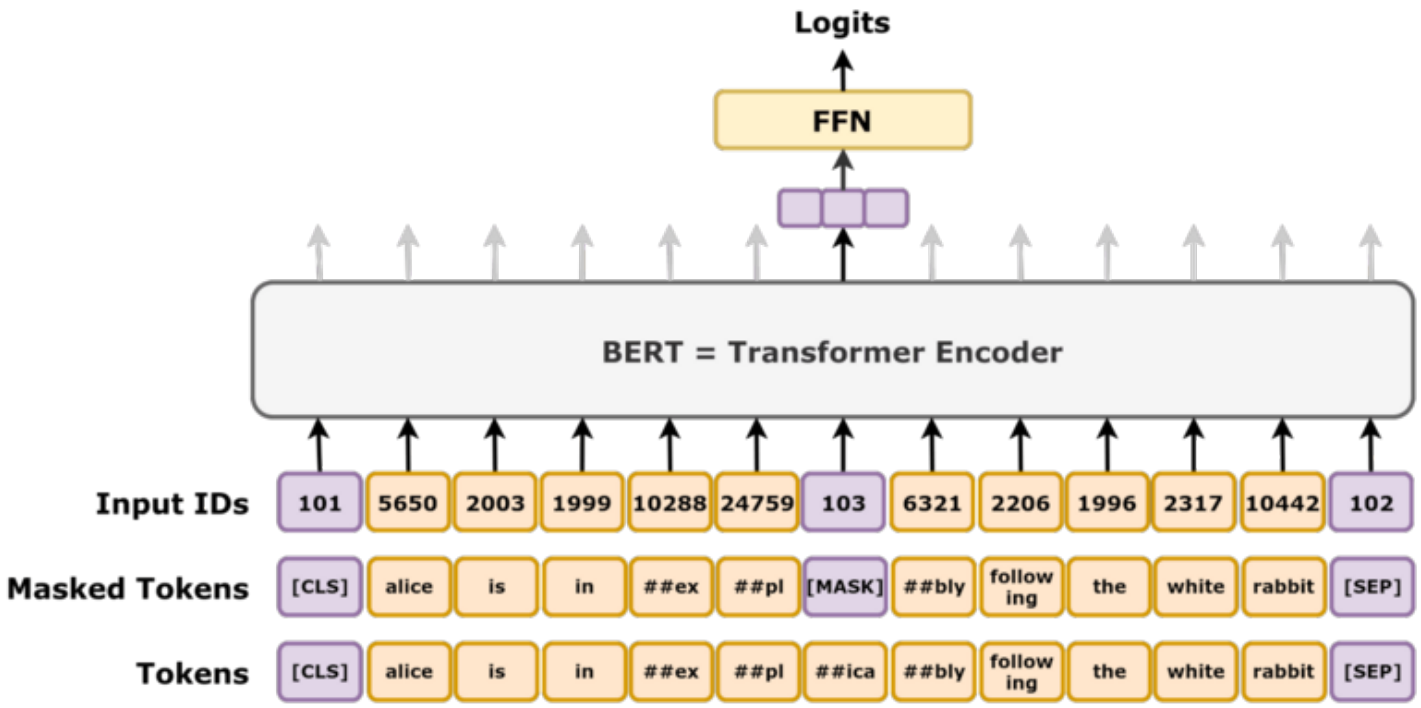
- **Masked language modeling (MLM)** El 15% de los tokens se seleccionan aleatoriamente para la tarea de predicción, el objetivo del entrenamiento es predecir los tokens enmascarados teniendo en cuenta su contexto.



Tareas de entrenamiento

BERT se pre-entrena simultáneamente en dos tareas:

- **Masked language modeling (MLM)**



Tareas de entrenamiento

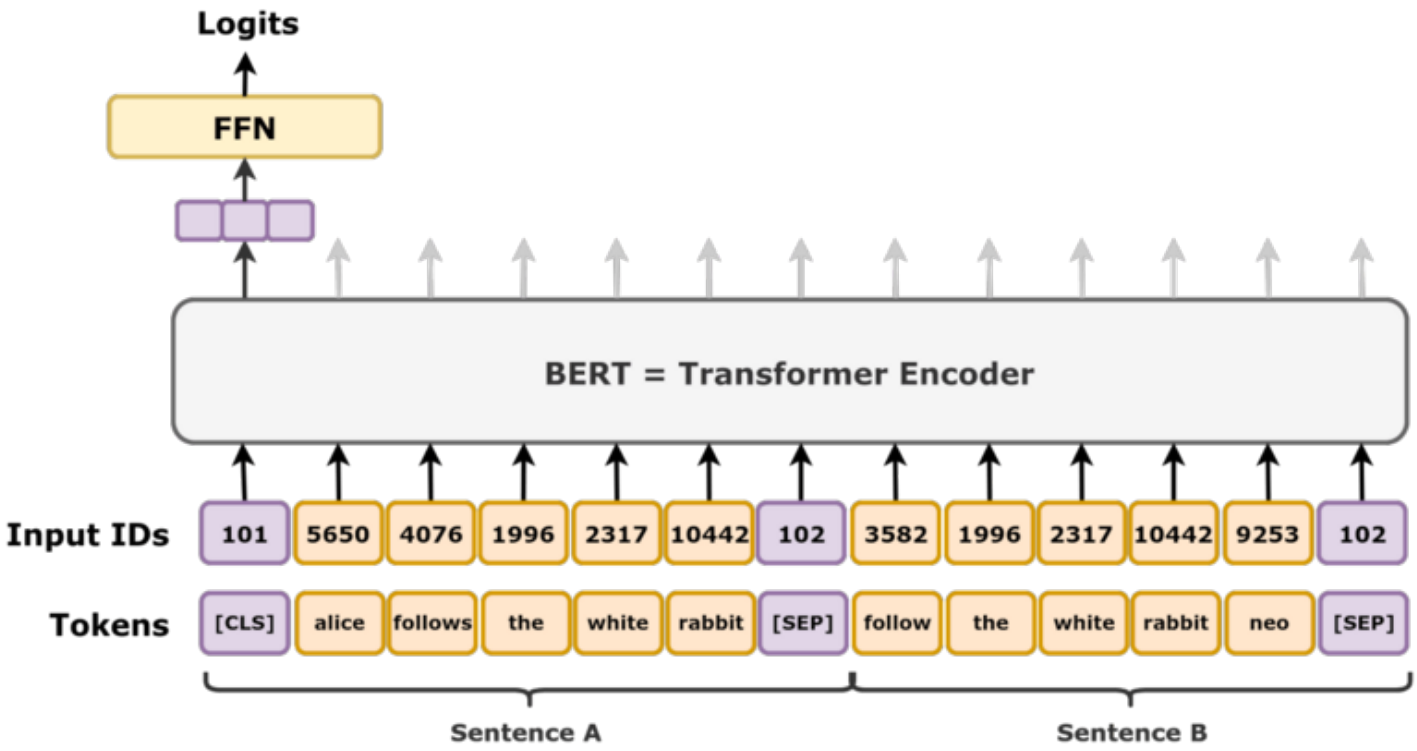
BERT se pre-entrena simultáneamente en dos tareas:

- **Next sentence prediction** Dados dos segmentos de texto, la tarea consiste en predicir si estos dos segmentos aparecieron secuencialmente en el corpus. El primer segmento comienza con un token especial `[CLS]` (classify). Los dos segmentos están separados por un token especial `[SEP]` (separate).

Tareas de entrenamiento

BERT se pre-entrena simultáneamente en dos tareas:

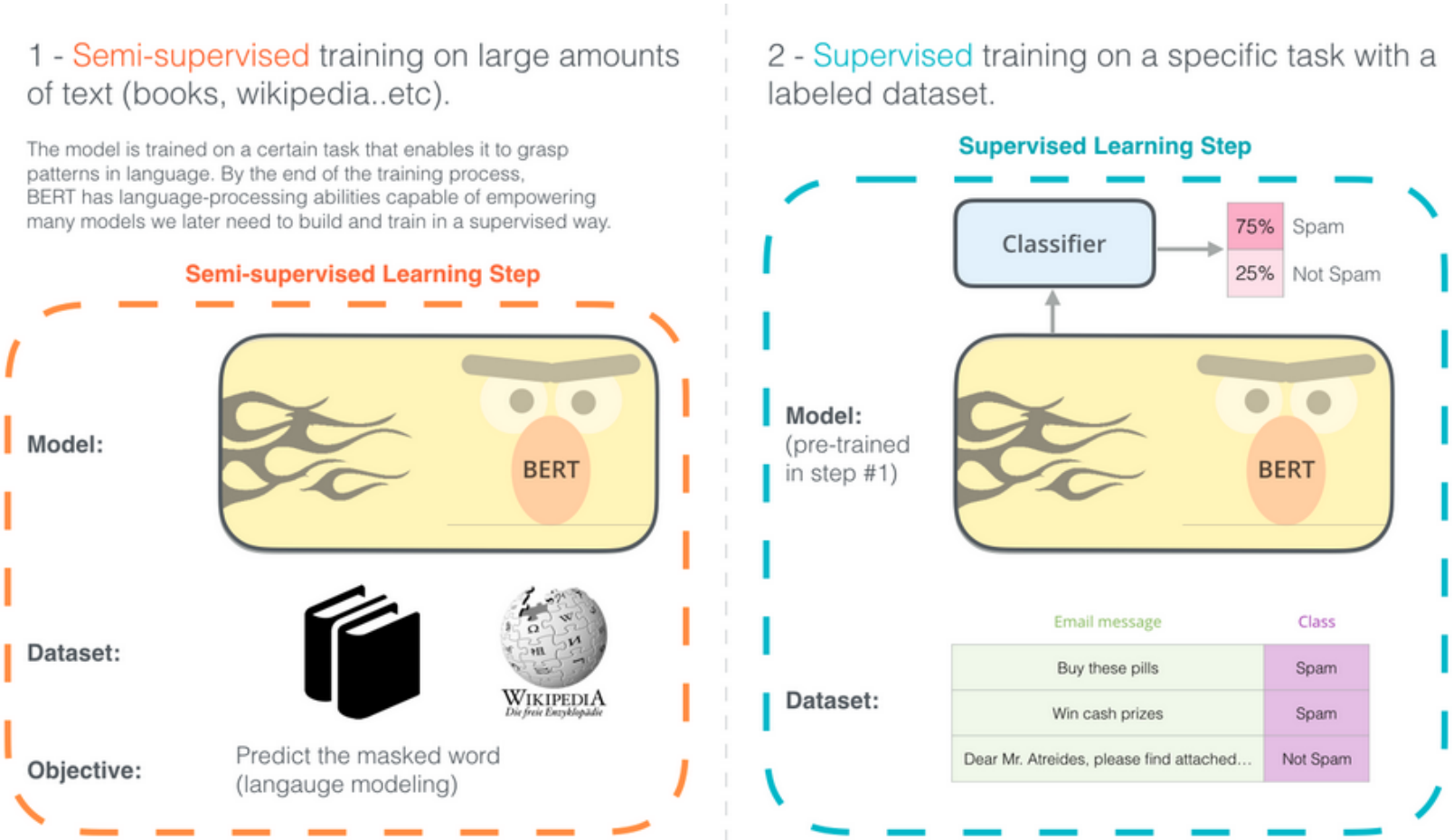
- **Next sentence prediction**



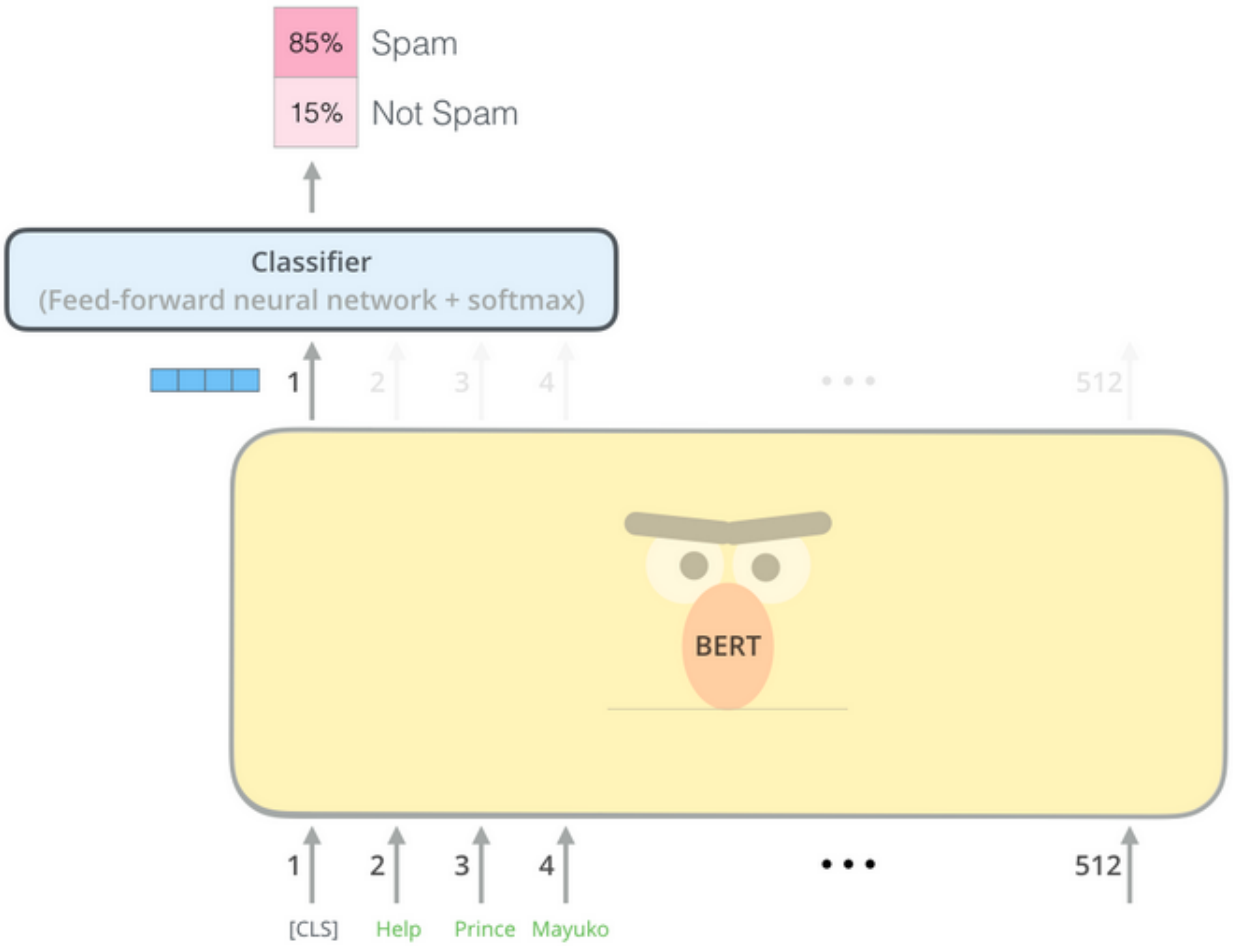
Entrenamiento

- $\texttt{BERT}_{\texttt{BASE}}$ se entrenó en 16 chips TPU y tardó 4 días, con un costo de $\sim$ 500 USD.
- $\texttt{BERT}_{\texttt{BASE}}$ se entrenó en 64 chips TPU y tardó 4 días.

Pre-training and Fine-tuning



Fine-Tuning


85%
Spam
15%
Not Spam
Classifier
(Feed-forward neural network + softmax)
1
2
3
4
512
BERT
1
2
3
4
512
[CLS]
Help
Prince
Mayuko

¿Dónde encontrar modelos BERT?

Hugging Face

Hugging Face, Inc. es una empresa conocida por su biblioteca de transformadores creada para aplicaciones de NLP y su plataforma que permite a los usuarios compartir datasets y modelos de ML. Su campo de acción principal es el NLP, pero también se centra en otras áreas del ML: CV, el aprendizaje por refuerzo y el aprendizaje supervisado.

repositorio

Limitaciones

- La tarea MLM es eficaz para tareas de comprensión; no es ideal para tareas generativas debido a su diseño bidireccional y no autorregresivo:
  - **No autorregresivo**: BERT es intrínsecamente no autorregresivo, lo que significa que no está diseñado para generar tokens secuencialmente de principio a fin.
  - **Dependencia bidireccional del contexto** BERT se basa en la información de ambas direcciones en una frase, lo que es beneficioso para la comprensión pero restrictivo en tareas que requieren la generación de token a token de izquierda a derecha, como la conversación o la escritura narrativa.
- Aun cuando los datos de entrenamiento utilizados pueden caracterizarse como neutros, este modelo puede tener predicciones sesgadas:

Modern Bert

Trained on 2 trillion tokens with a native 8192 sequence length

- $\text{ModernBERT}_{\text{BASE}}$ (149 million parameters)
- $\text{ModernBERT}_{\text{LARGE}}$ (395 million parameters)

Recordamos el tamaño de los modelos antiguos:

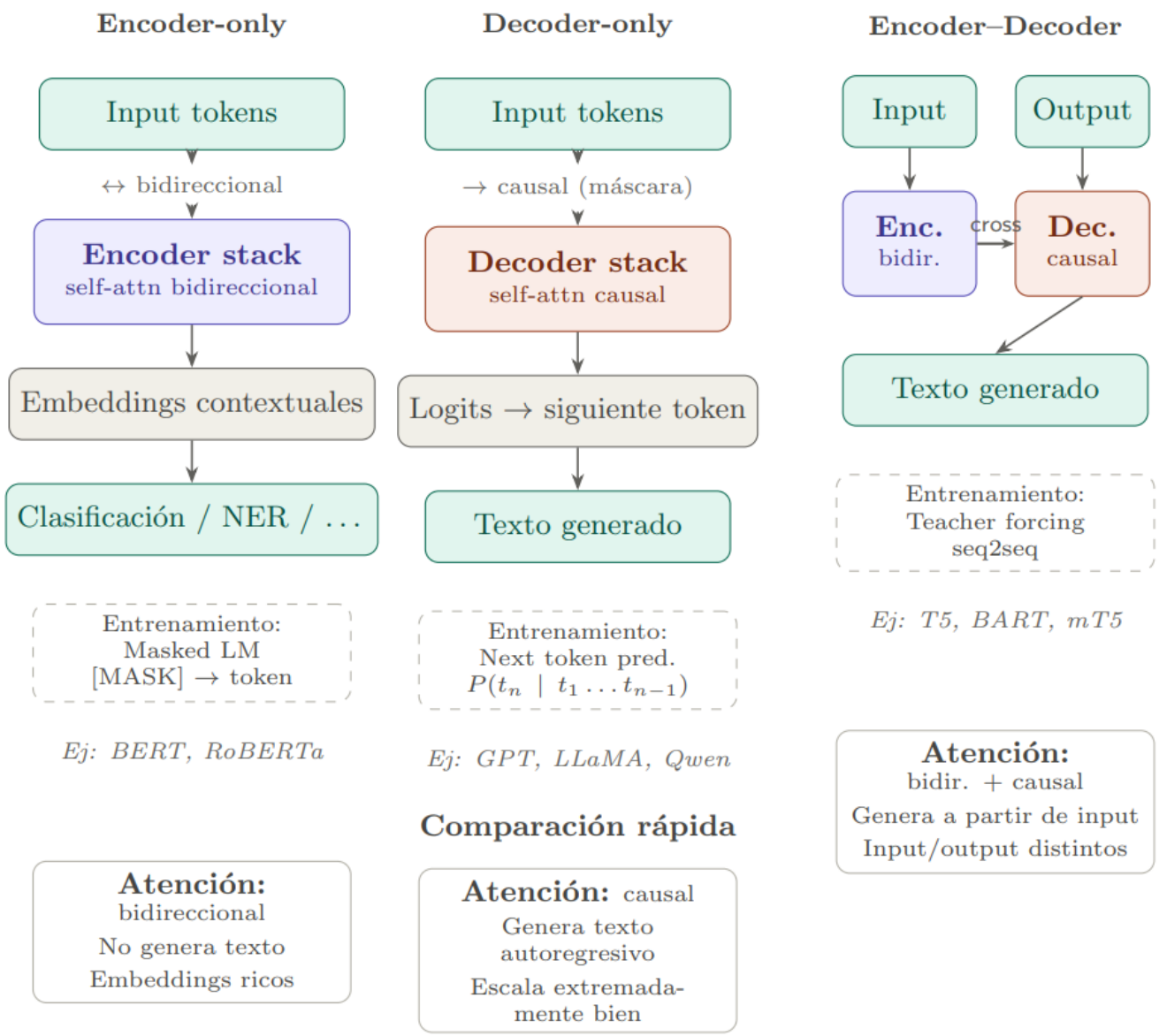
- $\text{BERT}_{\text{BASE}}$ (110 million parameters)
- $\text{BERT}_{\text{LARGE}}$ (340 million parameters)

Original Paper

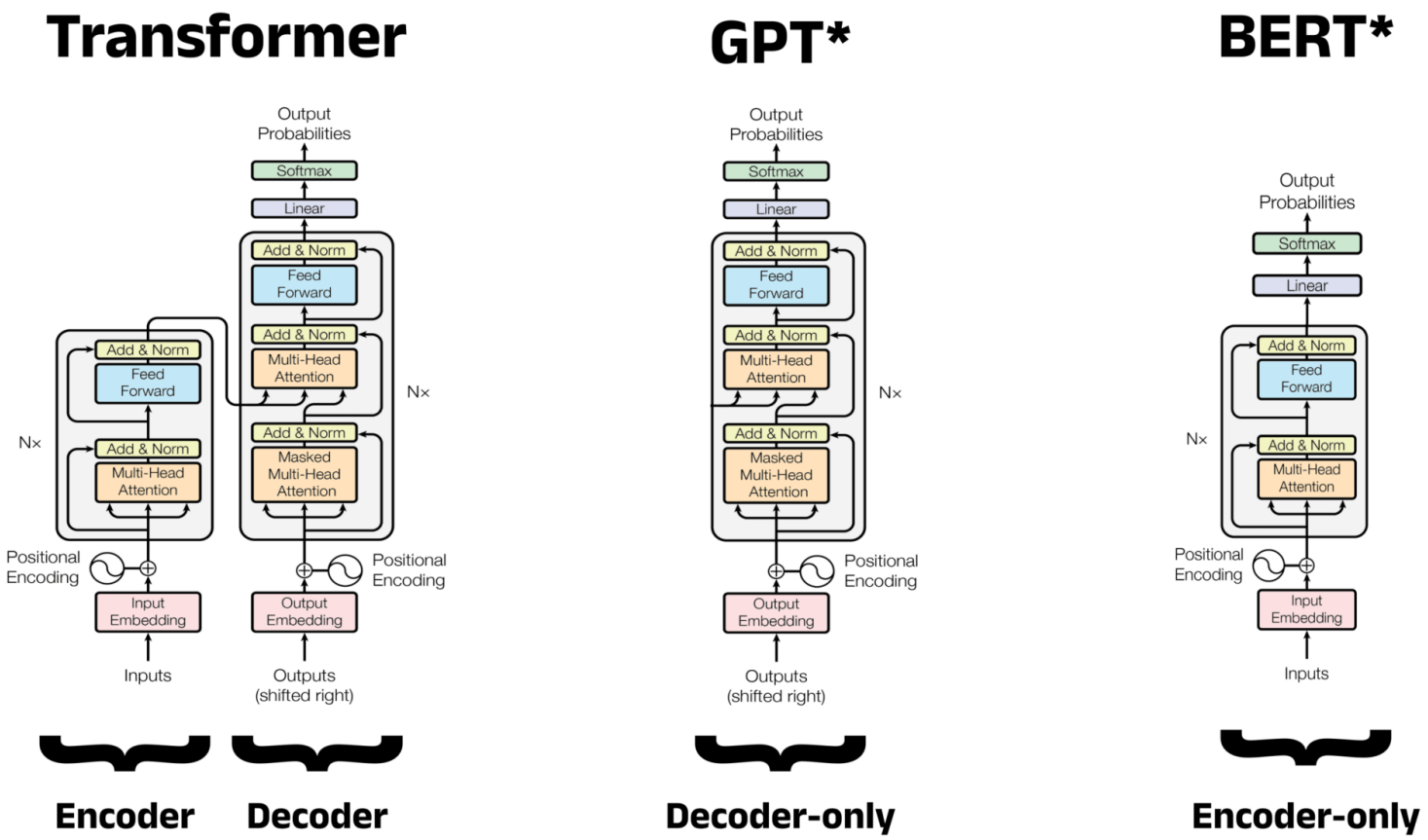
Section 3

Transformers / BERT / LLMs

Transformers / BERT / LLMs



Transformers / BERT / LLMs



*Illustrative example, exact model architecture may vary slightly

Source

Section 4

Large Language Models

Large Language Models

Los LLM (Large Language Model) son redes neuronales muy grandes, y profundas, que toman textos en lenguaje natural como entrada y producen otros textos en lenguaje natural como salida.

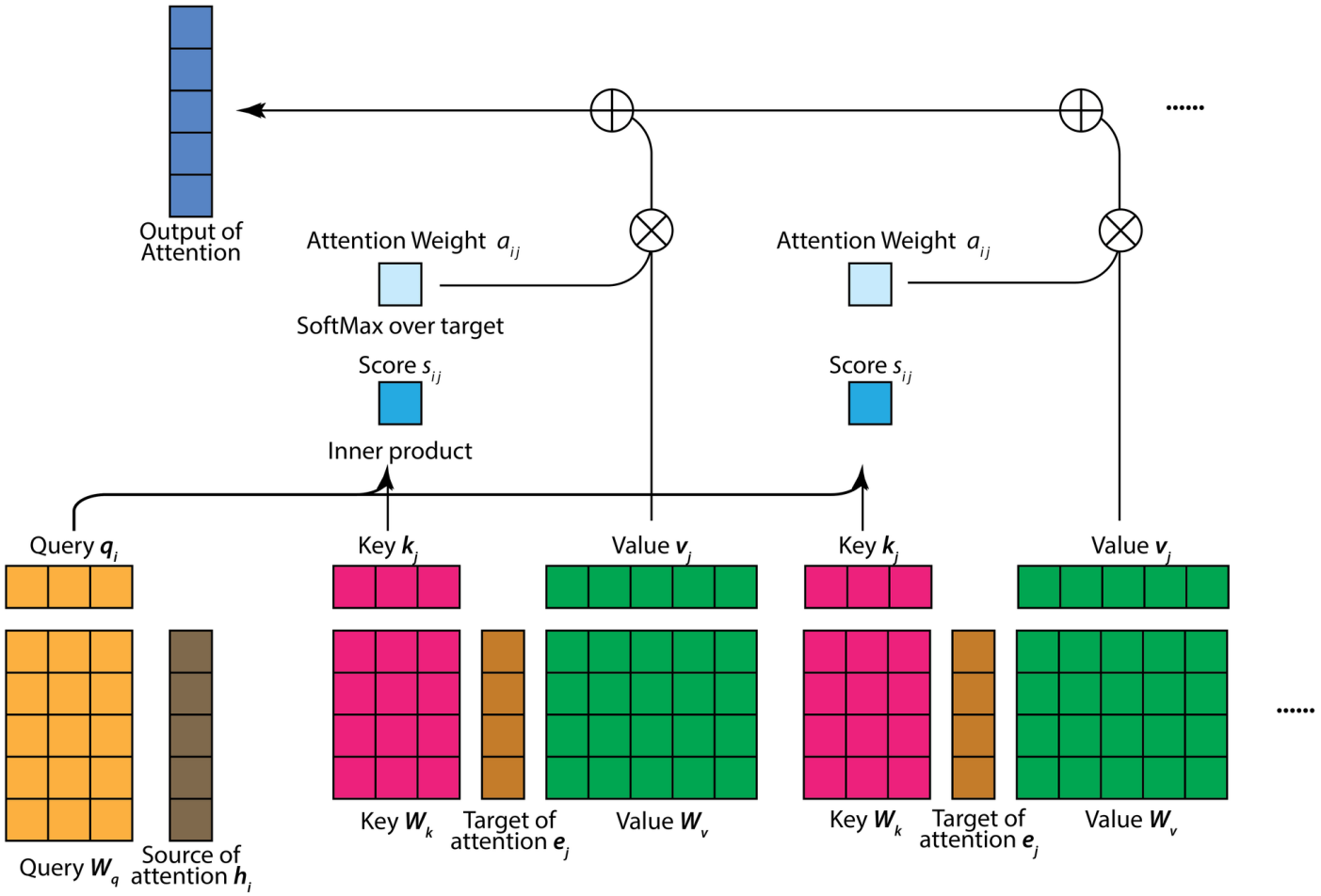
Es un tipo de modelo avanzado de IA diseñado para comprender, generar e interactuar con el lenguaje humano. Estos modelos son *grandes* porque se entrenan con grandes cantidades de texto y el modelo tiene grandes cantidades de parámetros, lo que les permite realizar una amplia gama de tareas relacionadas con el lenguaje natural, como responder preguntas, redactar, traducir, etc.

Algunos LLMs

- GPT (OpenAI)
- Gemini (Google)
- Gemma (Google)
- Llama (Large Language Model Meta AI, Meta)
- Claude (Anthropic)
- Mixtral (Mistral AI)
- Phi-3 (Microsoft)

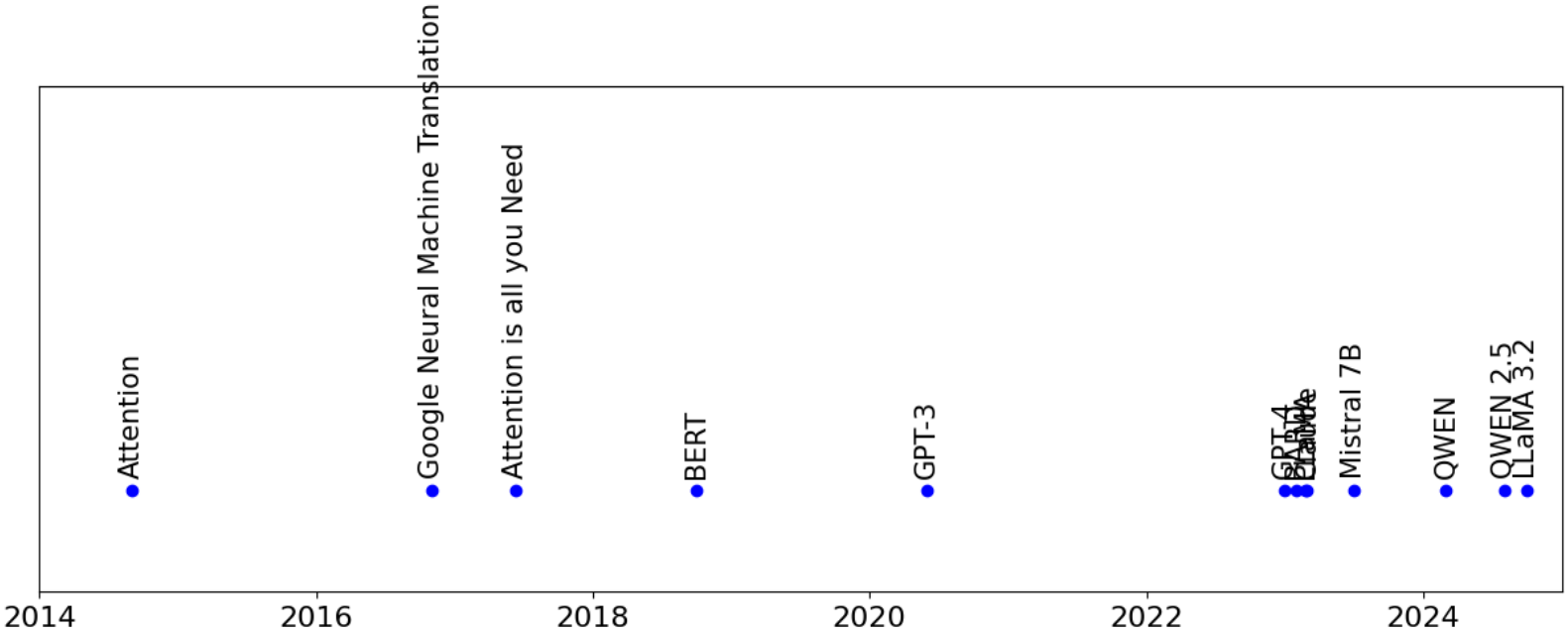
Huggingface 1, Huggingface 2

El mecanismo de atención: Reprise



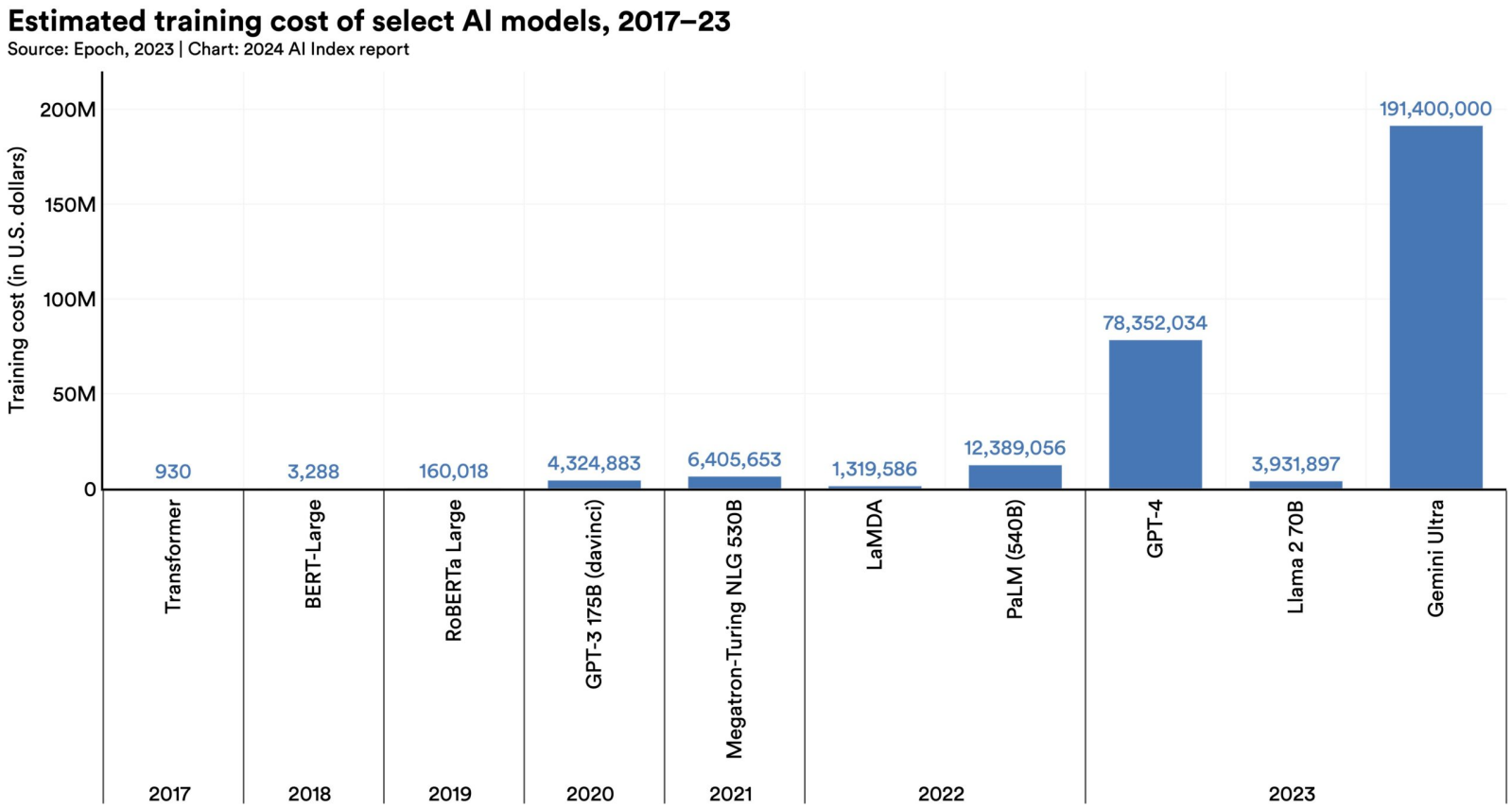
Source, LlaMA, Mistral 7b

Evolución: Timeline



Lista

Evolución: Costo de entrenamiento



Clasificación

Algunos consideran que sólo los modelos más grandes, con billones de parámetros, son verdaderos LLM, mientras que los modelos más pequeños, como DistilBERT, se consideran simples modelos de NLP. Otros incluyen modelos más pequeños, pero potentes, en la definición de LLM, también como DistilBERT.

- Los SLM tienen un número de parámetros que oscilan entre unos pocos millones y unos pocos billones.
- Los LLMs tienen un número de parámetros que oscilan entre pocos billones a algunos trillones de parámetros.

Key terms to know

- Embeddings
- Attention
- Transformers
- Prompting, Prompt Engineering
- RLHF (Reinforcement Learning with Human Feedback)
- Context size
- Quantization
- Zero-shot y Few-shot learning
- RAG
- Hugging Face, LangChain, Ollama

Consideraciones

El despliegue de LLMs requiere una consideración de diversos factores técnicos

- Requisitos previos de hardware y software:
  - Hardware: El despliegue de LLMs, especialmente los de gran escala como GPT-3 o LlaMA, requiere grandes recursos computacionales. Esto incluye GPUs o TPUs de alto rendimiento, amplia memoria (RAM) y almacenamiento sustancial para manejar grandes conjuntos de datos.
  - Software: El stack de software incluye marcos de ML (como PyTorch), herramientas de contenerización (como Docker) y sistemas de orquestación (como Kubernetes) para gestionar y escalar los despliegues de manera efectiva.

Consideraciones

El despliegue de LLMs requiere una consideración de diversos factores técnicos

- Despliegue en la nube frente a despliegue local:
  - Despliegue en la nube: Proporciona escalabilidad, flexibilidad y menores costos iniciales. Ofrece acceso a infraestructura y servicios avanzados sin necesidad de hardware físico. Proveedores populares como AWS, Google Cloud y Azure ofrecen servicios especializados de IA y ML para la implementación de LLMs.
  - Despliegue *in site*: Ofrece un mayor control sobre los datos y la infraestructura, esencial para organizaciones con estrictas necesidades de privacidad y seguridad de datos. Sin embargo, requiere una inversión inicial significativa y un mantenimiento continuo de hardware y software.

Subsection 1

Conceptos Importantes

Context Size

La ventana de contexto, **context size**, de un LLM es el número de tokens que el modelo puede tomar como entrada a la hora de generar respuestas.

Mayores *context size* aumentan la capacidad de aprendizaje en contexto de las instrucciones. Proporcionar más ejemplos y/o ejemplos más grandes como *prompt*, permite que al LLM dar una mejor respuesta. Por ejemplo, proporcionar al LLM información contextual que no estaba disponible en el momento en que se le formó para contestar información reciente.

Ejemplo de tokenización, How Language Models Use Long Contexts

Prompt Engineering

Un **prompt** es un texto, en lenguaje natural, que pide a un LLM que realice una tarea específica.

El **Prompt Engineering** es la práctica de diseñar y perfeccionar instrucciones de entrada para interactuar eficazmente con LLMs. El objetivo es extraer del modelo el resultado deseado, ya sea generar texto, responder preguntas, clasificar datos o realizar otras tareas.

Ejemplos

Where to purchase a shirt

You are a sales assistant for a clothing company. A user, based in Alabama, United States, is asking you where to purchase a shirt. Respond with the three nearest store locations that currently stock a shirt.

Otro ejemplo

Subsection 2

LLMs como modelos de Machine Learning

Definiciones clave

Al igual que los modelos tipo BERT, podemos usar los LLMs como modelos de Machine Learning.

- **Fine-tuning:** Adaptación de un modelo preentrenado a una tarea específica mediante entrenamiento con datos etiquetados.
- **Prompting (zero/few-shot):** Uso de instrucciones en lenguaje natural para guiar al modelo sin modificar sus parámetros; ideal para tareas rápidas o sin datos etiquetados.
- **PEFT (Parameter-Efficient Fine-Tuning):** Técnicas que ajustan solo una pequeña parte del modelo, reduciendo costos computacionales y de almacenamiento, y permitiendo adaptar modelos grandes sin reentrenarlos completamente. Implementación

Comparación entre métodos de clasificación

Criterio	**Fine-tuning**	**Prompting**	**PEFT (Híbrido)**
Datos etiquetados	Necesarios	No necesarios	A veces necesarios
Recursos técnicos	Altos	Medios/Bajos	Medios
Escalabilidad	Alta	Variable	Alta
Personalización	Alta	Media/Alta	Alta
Flexibilidad	Baja/Media	Alta	Alta

¿Dónde están los modelos?

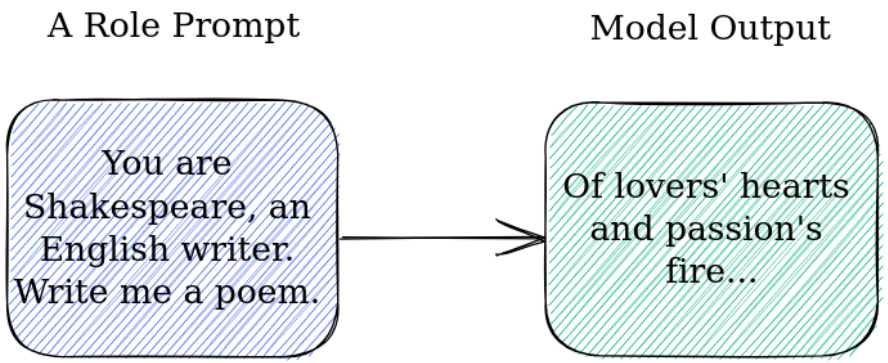
- Hugging face

- Ollama

- LangChain

Role Prompting

Rrole prompting es una técnica utilizada en el *Prompt Engineering* para guiar al LLM a abordar una pregunta o problema asumiendo un papel, personaje o punto de vista específico.



Personas in System Prompts Do Not Improve Performances of LLM

Ejemplos

Section 5

Agentes

Section 6

Agentes

Agentes

Un **agente** es un sistema que combina un LLM con herramientas externas, memoria y un bucle de razonamiento para actuar de manera autónoma sobre un entorno. A diferencia de un modelo que solo predice el siguiente token, un agente *decide* qué acción ejecutar, observa el resultado y determina si debe continuar o detenerse.

ReAct: Synergizing Reasoning and Acting in Language Models

Agentes

La diferencia clave respecto a RAG es la orquestación: mientras RAG recupera contexto y responde en un solo paso, un agente puede encadenar múltiples operaciones, corregirse a sí mismo y adaptar su estrategia según los resultados intermedios.

ReAct: Synergizing Reasoning and Acting in Language Models

¿Qué limitante abordan?

Los LLMs, incluso con RAG, operan como *cerebros sin manos*: no pueden ejecutar código, consultar APIs en tiempo real, interactuar con bases de datos ni modificar archivos. Sus conocimientos son estáticos y su respuesta es unívoca.

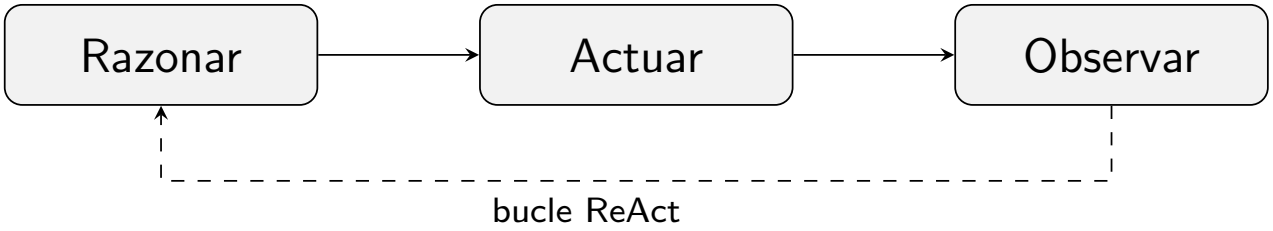
¿Qué limitante abordan?

Las limitaciones que un agente resuelve:

- **Iteración**: RAG responde una vez; un agente puede investigar, reintentar y refinar.
- **Verificación**: no asume que su primera respuesta es correcta; puede contrastarla contra datos externos.
- **Composición**: combina operaciones heterogéneas (buscar $\rightarrow$ calcular $\rightarrow$ buscar con el resultado).
- **Manejo de imprevistos**: ante un error de herramienta o una respuesta inesperada, ajusta su plan.

Tipos de agentes

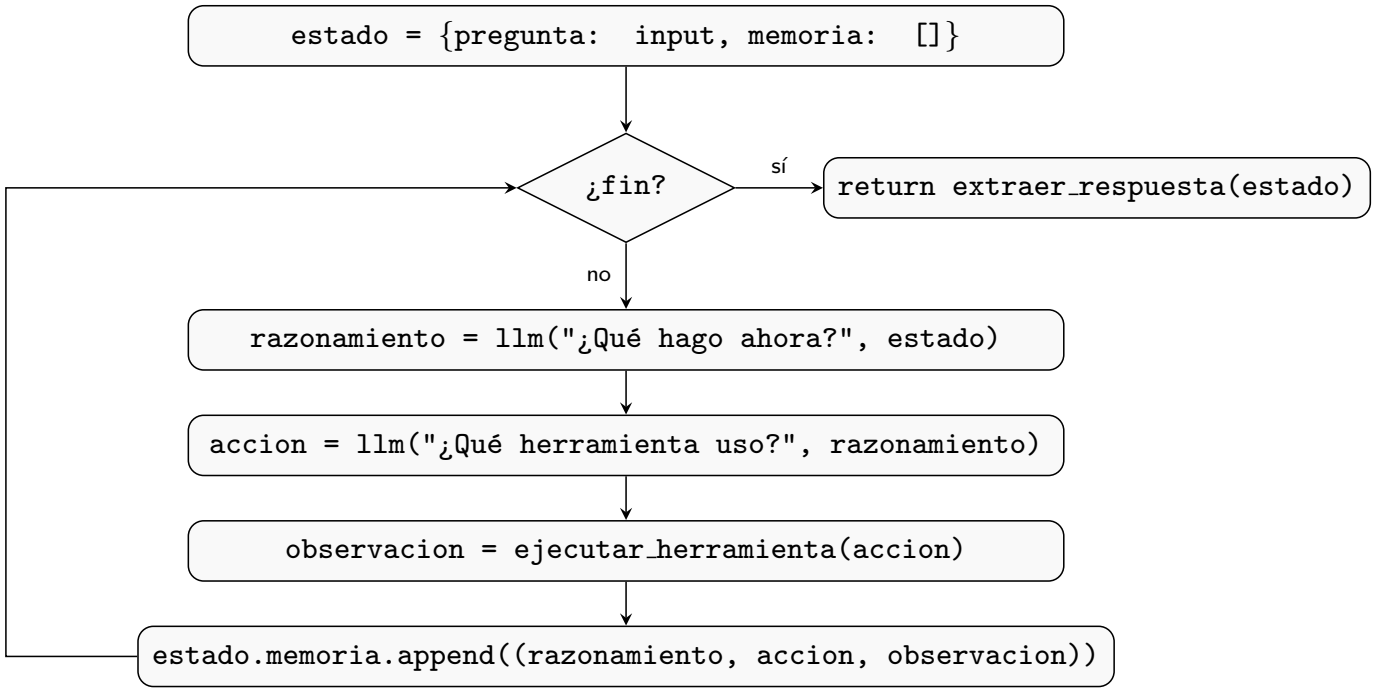
- **ReAct** (Reasoning + Acting): alterna pasos de razonamiento y acción. Es la arquitectura más común y versátil.



- **Plan-and-Execute**: genera un plan completo antes de ejecutar cualquier herramienta. Menos adaptable, más predecible.
- **Reflexión**: evalúa su propio desempeño y ajusta su comportamiento en ejecuciones posteriores.
- **Multi-agente**: varios agentes especializados colaboran, cada uno con herramientas y objetivos propios.

ReAct, Reflexion

Pseudocódigo de un agente ReAct



Ejemplo ReAct

Query: *¿Cuál fue el costo de entrenamiento de GPT-4 en dólares por hora de cómputo, si sabemos que duró 100 días?*

[Pensar] Necesito el costo total de entrenamiento de GPT-4. No lo sé de memoria. Debo buscarlo.

↓

[Actuar] herramienta: RAG, query: "costo entrenamiento GPT-4 2024"

↓

[Observar] Documento recuperado: "El entrenamiento de GPT-4 costó aproximadamente 100 millones de dólares."

↓

[Pensar] Tengo el costo total: 100M USD. Duración: 100 días. Necesito calcular USD/hora.

↓

[Actuar] herramienta: Python, código: "100_000_000 / (100 * 24)"

↓

[Observar] Resultado: 41666.67

↓

[Pensar] Tengo la respuesta: 41,667 USD por hora. Puedo responder.

↓

[Respuesta] El costo fue de aproximadamente 41,667 dólares por hora de cómputo.

Aplicaciones

- **Agente de investigación**: busca papers, extrae datos, ejecuta código para reproducir resultados, redacta comparativa.
- **Agente de análisis de datos**: recibe un dataset, detecta desbalance, genera código de augmentación, ejecuta, evalúa métricas, itera.
- **Agente de debugging**: recibe un error en finetuning, consulta logs y documentación, propone y aplica correcciones, verifica.
- **Agente multi-herramienta**: traduce texto, evalúa calidad con métrica; si el resultado no alcanza un umbral, reintenta con otro prompt.

Frameworks y herramientas

Programáticos	Ligeros	Low-code / Visual
LangChain / LangGraph	PydanticAI	n8n
LlamaIndex	SmolAgents	Dify
	Agno	Coze

Criterios de elección: ¿se necesita un grafo de estados explícito? ¿colaboración entre agentes? ¿control fino del bucle? ¿integración con sistemas existentes? La complejidad debe justificarse por la tarea.

LangChain, LlamaIndex, SmolAgents, n8n